OPERATING SYSTEMS – ASSIGNMENT 1

Name: S. Sharvesh **Register No:** 192525458

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems (4 Marks)

Evaluate registers, cache, main memory, SSD, hard disk and network storage with typical access times and sizes; where MRI images, patient records and monitor readings should be kept; programmed I/O, interrupt-driven I/O and DMA; and caching for record lookup.

Alternative Concept and Complexity Analysis: If a 500 MB MRI image used interrupt-driven I/O with one interrupt per word instead of DMA, evaluate interrupt count, CPU time and buffers.

Evidence Expected: Justified data placement and I/O method using typical access times.

ANSWER

9.1 Memory hierarchy and typical values

The following are approximate order-of-magnitude values. Actual latency and capacity vary by processor, memory technology, storage device and network.

9.2 Data placement

MRI images: acquire into scanner/server RAM buffers, process using RAM and CPU cache, store active studies on high-speed SSD/NVMe, then move older studies to larger archive or network storage according to retention policy.

Patient records: keep the active database working set and indexes in RAM/cache; persist transaction logs and database files on redundant low-latency SSD storage; replicate and back up to separate failure domains.

Monitor readings: keep immediate sensor values and alarm state in RAM/registers for deterministic response; persist selected trends and audit events to local durable storage and forward them to the central system asynchronously.

9.3 Programmed I/O, interrupt-driven I/O and DMA

For MRI scanning, DMA is preferred. The CPU configures the transfer, the DMA engine moves a block into memory, and an interrupt reports completion. The OS must use appropriate buffer pinning/mapping, cache coherency and access controls.

Caching improves record lookup because frequently accessed records and indexes may be served from RAM rather than slower SSD or network storage. Cache invalidation and consistency are important: stale clinical data must not be presented as current. Use transaction consistency, versioning and appropriate cache time-to-live rules.

9.4 Alternative: one interrupt per word for a 500 MB MRI image instead of DMA

Assume one word is 4 bytes and 500 MB is 500 × 1,000,000 bytes. The number of words is 500,000,000 / 4 = 125,000,000. Interrupt-driven transfer with one interrupt per word therefore generates about 125 million interrupts. If each interrupt and handler costs even 1 microsecond, the CPU spends about 125 seconds servicing interrupts; at 5 microseconds it spends about 625 seconds. These are illustrative calculations, not measured hardware values.

DMA can transfer the same image using a small number of block descriptors and completion interrupts. The CPU configures the transfer and processes data in parallel. Interrupt-per-word transfer requires repeated CPU handling and may need a staging buffer; DMA needs suitably sized pinned/contiguous or mapped buffers and descriptor rings. For large images, DMA has dramatically lower CPU overhead.

QUESTION FROM THE PROVIDED ASSIGNMENT

Level | Typical access time | Typical capacity | Hospital use

CPU registers | About 0.3–1 ns (one/few cycles) | Bytes to a few KB per core | Current instructions, addresses and immediate calculations.

CPU cache (L1/L2/L3) | About 1–20 ns depending on level | Tens of KB to tens of MB | Frequently used code, database indexes and hot data.

Main memory (RAM) | About 50–100 ns | GBs to TBs in servers | Active database working set, MRI processing buffers and OS processes.

NVMe / SSD | About 10–100 microseconds for common I/O | Hundreds of GB to many TB | Patient database files, recent MRI images, logs and fast backups.

Hard disk | About 5–15 ms random access | TBs to tens of TBs | Lower-cost archive and bulk backup where latency is acceptable.

Network storage | Sub-millisecond to many milliseconds or more | Scalable | Replicated records, shared images and off-site/cloud backup; latency depends on network.

Method | Operation | Evaluation / suitable use

Programmed I/O (polling) | CPU repeatedly checks device status and transfers data. | Simple but wastes CPU time; suitable only for small or tightly controlled transfers.

Interrupt-driven I/O | Device interrupts CPU when ready; CPU services the event and transfers data. | Avoids continuous polling, but many interrupts can consume CPU time. Suitable for moderate event rates.

Direct Memory Access (DMA) | DMA controller transfers blocks between device and memory with limited CPU involvement; completion is signalled by interrupt. | Best for high-volume MRI image transfers because the CPU can process other work while data moves.